

CSS 텍스트, 폰트와 JS 기초

오늘의 강의 학습 요약

본 수업에서는 CSS의 폰트 지정 방식(제네릭 패밀리/실제 폰트 패밀리, 폰트 단축 표기, 구글 폰트 적용)과 텍스트 관련 속성(정렬, 변환, 들여쓰기, 간격 등)을 정리하며 CSS 단원을 마무리했습니다. 또한 실무에서는 순수 CSS보다 Bootstrap/Tailwind 같은 프레임워크를 많이 활용한다는 점과, Flexbox, Grid의 역할 차이(1축 vs 2축 레이아웃)를 개념 수준으로 연결했습니다.

이후 자바스크립트로 전환하여 JavaScript/ECMAScript 표준화 배경, 브라우저 엔진(V8)과 Node.js의 등장 의미, HTML에서 스크립트 포함 방식, 콘솔 디버깅을 다뤘습니다. 이어서 데이터 타입(기본형/참조형), truthy/falsy, typeof, 변수 선언(var/let/const)과 스코프, 자동 형 변환, 비교 연산자 == vs ===, 논리 연산자의 단락 평가 결과, 참조 전달 특성, 상수에서의 "주소 기반 불변성"과 프론트엔드 프레임워크의 불변 객체 개념까지 폭넓게 정리했습니다. 마지막으로 내장 객체를 "데이터/브라우저(BOM)/문서(DOM)"로 구분하고, BOM 객체 및 window의 전역 객체 성격, 타이머, 창 제어 API, 문자열 백틱 템플릿 리터럴, 배열 메소드, JSON(객체 리터럴) 개요를 다뤘습니다.

1. CSS 폰트와 텍스트 속성

폰트는 제네릭 패밀리(카테고리)와 실제 폰트 패밀리(글꼴 이름)로 구분되며, `font-family` 는 실제 폰트들을 나열한 뒤 마지막에 제네릭 패밀리를 폴백으로 둡니다.

CSS 폰트 관련 핵심 속성은 `font-family` , `font-style` , `font-weight` , `font-size` 로 정리됩니다. 글꼴 지정은 “실제 폰트 패밀리 → 폴백 폰트 패밀리 → 마지막 제네릭 패밀리” 순으로 나열하는 방식이 일반적입니다.

`font-family` 에서 제네릭 패밀리는 글꼴 “카테고리”이며, 실제 폰트 패밀리는 그 카테고리 안의 구체 글꼴 이름입니다. 예를 들어 Serif(꼬리 있음), Sans-serif(꼬리 없음), Monospace(폭이 고정된 글자)가 제네릭 패밀리의 대표 예시입니다.

폰트는 로컬(사용자 PC)에 실제 글꼴이 설치되어 있어야 적용됩니다. 지정한 글꼴이 없다면 다음 후보로 넘어가며, 모두 없으면 마지막 제네릭 패밀리로 브라우저 기본 설정 글꼴이 적용됩니다.

폰트 속성은 각각 개별로 지정할 수도 있지만, 실무에서는 단축 표기인 `font` 를 자주 사용합니다. 단축 표기는 순서가 중요하며 일반적으로 스타일 → 웨이트 → 사이즈 → 패밀리 순으로 작성합니다.

폰트 크기 단위는 예제로 `em` 을 사용했으나, 상대 단위 사용 시에는 `rem` 사용을 권장한다는 점이 강조되었습니다.

구글 폰트(Google Fonts)는 무료 웹폰트 제공 서비스이며, 원하는 글꼴을 선택해 `<link>` 임베드 코드를 `<head>` 에 추가하고, CSS `font-family` 에 제공된 폰트명을 그대로 사용하면 됩니다. 한글 폰트는 필터에서 Language를 한국어로 설정하고, Serif/Sans-serif 같은 분류를 함께 좁혀 선택하는 흐름으로 진행됩니다.

텍스트 관련 속성으로는 정렬 및 장식, 변환, 간격 제어가 포함됩니다. `text-align` 의 `justify` 는 양쪽 정렬이지만 가독성이 떨어질 수 있어 사용 맥락을 고려해야 합니다.

텍스트 속성 범주는 다음과 같이 묶어 이해할 수 있습니다.

범주	대표 속성	핵심 의미
정렬	<code>text-align</code>	<code>left/right/center/justify</code> 로 정렬 방식을 지정합니다.
장식	<code>text-decoration</code>	밑줄, 취소선 등 장식선을 지정합니다.
변환	<code>text-transform</code>	대문자/소문자 변환 등 글자 형태를 변환합니다.
들여쓰기, 간격	<code>text-indent</code> , <code>letter-spacing</code> , <code>word-spacing</code> , <code>line-height</code>	첫 줄 들여쓰기, 글자/단어/줄 간격을 제어합니다.

CSS 레이아웃 확장 개념으로 Flexbox와 Grid가 간단히 언급되었습니다. Flexbox는 한 축(가로 또는 세로) 중심 정렬, 배치에 강하고, Grid는 2축(가로, 세로)을 함께 다뤄 더 구조적인 레이아웃에 적합합니다.

실무 스타일링은 순수 CSS만으로 하기보다 Bootstrap, Tailwind 같은 프레임워크를 쓰는 경우가 많다고 정리되었습니다. 이는 JDBC 대신 MyBatis/JPA를 쓰는 것처럼 생산성을 높이는 도구라는 비유로 연결되었습니다.

핵심 키워드

#font-family

#제네릭 패밀리

#font 단축표기

#구글 폰트

#text-align

2. 자바스크립트 개요와 표준화

자바스크립트는 브라우저에서 실행되는 동적 언어이며, 브라우저 벤더 차이로 인해 표준 (ECMAScript)이 필요했고 V8 엔진과 Node.js로 실행 환경이 브라우저 밖까지 확장되었습니다.

자바스크립트는 1995년 Netscape에서 시작되었고, 초기 명칭은 LiveScript였으나 Java의 인기에 편승하여 JavaScript로 이름이 바뀌었다는 배경이 설명되었습니다. Java와 JavaScript는 이름만 유사할 뿐 언어적으로는 별개입니다.

브라우저마다 구현이 달라 동일 문법 보장이 어려웠기 때문에 표준화가 필요했으며, HTML/CSS/SQL이 표준을 갖게 된 이유와 동일한 맥락으로 ECMAScript 표준이 등장합니다. 본 수업에서는 ES6(2015)을 표준화 전환점으로 정리했습니다.

자바스크립트는 브라우저에서 실행되며, 브라우저마다 자바스크립트를 실행하는 엔진이 존재합니다. 특히 Chrome의 V8 엔진이 중요하며, V8이 브라우저 밖으로 분리되면서 Node.js 플랫폼이 만들어져 자바스크립트가 서버 등 다양한 환경에서 실행 가능해졌다는 역사적 의미가 강조되었습니다.

HTML에서 자바스크립트를 포함하는 방식은 크게 3가지로 정리됩니다.

방식	위치/형태	특징
내부 스크립트 (Head)	<code><head><script>...</script></head></code>	문법적으로 가능하며 실행 시점 차이는 추후 설명 대상입니다.
내부 스크립트 (Body)	<code><body><script>...</script></body></code>	Body 내부 어느 위치든 가능하며 위치에 따라 실행 시점이 달라집니다.
외부 파일	<code><script src="..."></script></code>	재사용성이 높아 실무에서 가장 많이 사용하는 방식입니다.

자바스크립트 주석은 자바와 동일하게 한 줄(//)과 여러 줄(/* */)을 사용합니다. 또한 대소문자를 구별한다는 점이 반복적으로 강조되었습니다.

디버깅/출력은 `alert()` 와 `console.log()` 를 중심으로 진행되었습니다. `console.log()` 는 브라우저 개발자 도구(F12)의 Console 탭에서 확인하며, 자바의 `System.out.println` 과 유사한 역할로 설명되었습니다.

`console.log` 는 쉼표로 여러 값을 함께 출력할 수 있다는 차이점이 언급되었습니다. 개발 시에는 콘솔 영역을 활성화하여 오류와 출력 결과를 확인하는 습관이 중요합니다.

핵심 키워드

#ECMAScript

#ES6

#V8

#Node.js

#console.log

3. 데이터 타입과 truthy/falsy, typeof

자바스크립트는 기본형과 참조형으로 나뉘며, 숫자는 `number` 로 통합되고 문자열은 `string` 으로 통합되며, 5가지 falsy 값과 `typeof` 의 문자열 반환 특성이 중요합니다.

자바스크립트 데이터 타입은 기본형(Primitive)과 참조형(Reference)으로 구분됩니다. 자바와 유사한 구조지만 타입 체계와 동작에서 차이가 존재합니다.

기본형에서 숫자는 정수/실수를 분리하지 않고 `number` 하나로 취급됩니다. 문자/문자열도 구분하지 않고 `string` 하나로 취급되며, 홀따옴표/쌍따옴표 모두 문자열 표현에 사용할 수 있습니다.

논리형은 `true/false` 가 기본이지만, 자바스크립트는 `true/false` 가 아닌 값도 조건식에서 논리값으로 평가될 수 있습니다. 특히 다음 5가지는 falsy로 취급되며, 나머지는 truthy로 취급됩니다.

구분	값	의미
falsy	0	숫자 0은 거짓으로 평가됩니다.
falsy	""	빈 문자열은 거짓으로 평가됩니다.
falsy	null	값이 없음을 나타내며 거짓으로 평가됩니다.
falsy	undefined	초기화되지 않음을 나타내며 거짓으로 평가됩니다.
falsy	NaN	숫자가 아님을 나타내며 거짓으로 평가됩니다.

`undefined` 는 변수를 선언했지만 초기화하지 않은 상태를 의미합니다. `null` 은 초기화는 했으나 “값이 없음”을 의도적으로 넣은 상태로 구분됩니다.

`NaN` 은 `Number.parseInt(...)` 등으로 숫자 변환을 시도했으나 변환 불가능한 문자열을 넣었을 때 발생하는 “Not a Number” 상태입니다. `Number` 의 첫 글자가 대문자인 점을 통해 자바스크립트에도 클래스/객체 형태(내장 객체)가 존재함을 연결해서 설명했습니다.

참조형 데이터는 대표적으로 객체(중괄호), 배열(대괄호), 함수, 클래스가 정리되었습니다. 특히 함수가 “데이터로 취급된다”는 점이 매우 중요한 특징으로 강조되었습니다.

`typeof` 연산자는 값의 타입을 “문자열”로 반환합니다. 참고로 `null` 의 `typeof` 결과가 `object` 로 나오는 점, 배열도 `object` 로 나오는 점, 함수는 `function` 으로 나오는 점이 예제로 확인되었습니다.

```
typeof 10          // "number"  
typeof "a"         // "string"  
typeof undefined   // "undefined"  
typeof null        // "object"  
typeof []          // "object"  
typeof function(){} // "function"
```

핵심 키워드

#기본형

#참조형

#falsy

#typeof

#NaN

4. 변수 var, let, const와 스코프

변수 선언은 `var` (구버전)과 `let` (ES6), 상수는 `const` (ES6)로 구분되며, `var` 는 함수 스코프, 중복 선언 가능, `let/const` 는 블록 스코프, 중복 선언 불가가 핵심입니다.

자바스크립트 변수는 데이터 타입을 선언부에 명시하지 않는 형태로 사용합니다. 동일 변수에 숫자, 문자열, 논리값 등 서로 다른 타입이 저장될 수 있어 유연하지만, 타입 안정성이 낮아 관리가 어려워질 수 있다는 점이 연결 설명되었습니다.

변수 선언 키워드는 `var` 와 `let` 이 있으며, 수업에서는 ES6 이후 표준 흐름에서 `let` 사용을 권장했습니다. 두 키워드는 중복 선언 가능 여부와 스코프 규칙이 다릅니다.

`var` 의 특징은 다음과 같이 정리됩니다.

- `var` 는 같은 스코프에서 변수명 중복 선언이 가능하여 실무 품질 측면에서 위험할 수 있습니다.
- `var` 는 블록 스코프가 아니라 함수 스코프를 따르므로, 함수가 아닌 블록의 내부에서 선언한 변수가 블록 밖에서도 접근될 수 있습니다.
- `var` 로 선언만 하고 초기화하지 않으면 값은 `undefined` 가 됩니다.
- `var` 는 레거시 코드에서 등장할 수 있으므로 읽을 수 있어야 합니다.

`let` 은 자바와 유사하게 블록 스코프를 따르며, 같은 스코프에서 중복 선언이 불가능합니다. `if/for` 같은 블록 안에서 선언된 `let` 변수는 블록 밖에서 접근할 수 없다는 점이 실습으로 확인되었습니다.

상수는 `const` 로 선언하며 ES6 문법입니다. `const` 는 `let` 과 동일하게 블록 스코프 및 중복 선언 불가 규칙을 가지며, 재할당이 금지됩니다. 또한 예외 처리로 `try/catch` 문법이 가능하되, 자바처럼 예외 타입을 명시하지 않고 변수로 받는 형태가 소개되었습니다.

프론트엔드 프레임워크(React/Vue)에서는 `const` 사용 빈도가 높다는 점이 강조되었고, 이는 불변성(주소 기반 변경 감지)과 연결된 실무 관례로 언급되었습니다.

핵심 키워드

`#var`

`#let`

`#const`

`#함수 스코프`

`#블록 스코프`

5. 형 변환, 비교, 논리 연산자 동작

더하기는 문자열 결합으로 동작하고 나머지 산술 연산은 숫자 변환 후 계산되며, 비교는 `===` 사용을 권장하고, `&&` / `||` 는 논리값이 아닌 실제 피연산자 값을 반환합니다.

자바스크립트는 자동 형 변환이 자주 발생합니다. 문자열 숫자와 숫자의 산술 연산에서 `+` 는 결합으로 동작하지만 `-` , `*` , `/` , `%` 는 문자열을 숫자로 변환하여 연산이 수행됩니다.

나눗셈 또한 자바와 차이가 있으며, 정수 나눗셈처럼 떨어지지 않고 실수 결과가 나올 수 있다는 점이 체크 포인트로 언급되었습니다.

비교 연산에서는 `==` (Equal)과 `===` (Identical/Strict Equal)의 차이가 핵심입니다. `==` 는 값만 비교하고, `===` 는 값과 타입을 함께 비교합니다. 실무 권장 방식은 `===` 사용입니다.

특히 `undefined` 비교는 `===` 를 사용해야 하며, `==` 로 비교할 경우 `null` 과도 같게 평가되는 문제가 발생할 수 있다는 점이 실습으로 확인되었습니다.

논리 연산자도 `&&` , `||` , `!` 자체는 자바와 유사하지만, 자바스크립트에서는 피연산자가 꼭 불리언일 필요가 없습니다. `truthy/falsy` 규칙에 따라 평가되며, `&&` / `||` 는 최종적으로 “선택된 피연산자 값”을 반환합니다.

`&&` 는 첫 값이 `falsy`면 첫 값을 반환하고, 첫 값이 `truthy`면 두 번째 값을 반환합니다. `||` 는 첫 값이 `truthy`면 첫 값을 반환하고, 첫 값이 `falsy`면 두 번째 값을 반환하는 규칙으로 정리되었습니다.

핵심 키워드

#자동 형변환

#===

#undefined 비교

#단락 평가

#&& || 반환값

6. 참조 전달과 상수의 주소 불변성

기본형은 전달, 대입 시 값이 복사되고, 참조형은 주소가 공유되어 변경 영향이 전파되며, `const` 는 “주소 재할당”을 막을 뿐 참조 대상의 요소 변경까지 막지는 않습니다.

기본형(Primitive)은 다른 곳으로 전달하거나 대입해도 원본 값이 보호되는 “값 복사” 특성을 가집니다. 반면 참조형(배열, 객체 등)은 주소가 전달되어, 전달받은 쪽에서 내부 요소를 변경하면 원본에도 영향이 생길 수 있습니다.

상수 `const` 에 대해 중요한 개념으로 “요소값 변경 가능”이 설명되었습니다. `const` 는 변수에 저장된 참조(주소)의 재할당을 금지하지만, 그 주소가 가리키는 객체/배열 내부의 요소 값 변경은 막지 않습니다.

즉, 자바스크립트가 변경 여부를 판단하는 관점은 “요소 값”이 아니라 “주소 값”이라는 관점으로 설명되었고, 이 개념이 React/Vue 등의 변경 감지 및 렌더링(불변 객체 패턴)과 연결된다고 강조되었습니다.

프레임워크에서 변경을 확실히 감지시키려면 기존 배열/객체의 요소만 바꾸는 것이 아니라, 새로운 배열/객체를 만들어 “새 주소”로 교체하는 방식이 필요하다는 불변성 개념이 예고되었습니다.

핵심 키워드

#기본형 복사

#참조 공유

#const

#주소값

#불변객체

7. 내장 객체: BOM, DOM, 데이터 객체

내장 객체는 데이터 관련 객체와 브라우저 관련 객체(BOM), 문서 객체(DOM)로 구분되며, BOM/DOC 객체는 브라우저 열림, 문서 렌더링 시 자동 생성되고 `window` 는 전역 객체로서 이를 포함합니다.

자바스크립트의 객체는 크게 내장 객체와 사용자 정의 객체로 구분되며, 사용자 정의 객체는 객체 리터럴(중괄호 키-값) 또는 클래스 문법으로 만들 수 있다고 정리되었습니다.

내장 객체는 크게 세 범주로 묶어 이해합니다.

범주	대표 객체	핵심 역할
데이터 관련	<code>String</code> , <code>Number</code> , <code>Date</code> , <code>Array</code> , <code>Boolean</code> , <code>Math</code> , <code>RegExp</code>	문자열/숫자/날짜/배열/수학 등 데이터 처리 기능을 제공합니다.
브라우저 관련 (BOM)	<code>window</code> , <code>screen</code> , <code>location</code> , <code>history</code> , <code>navigator</code>	브라우저 창, 화면, URL, 이동 기록, 브라우저 정보를 제공합니다.
문서 관련 (DOM)	<code>document</code>	렌더링된 HTML을 객체 모델(DOM 트리)로 접근, 조작합니다.

BOM 객체와 `document` 는 일반 객체와 달리, 브라우저가 열리거나 문서가 렌더링될 때 자동으로 생성됩니다. 개발자는 이를 `screen` , `location` , `history` , `navigator` , `document` 같은 이름으로 참조하여 속성/메소드를 사용할 수 있습니다.

`window` 는 브라우저에서 실행되는 자바스크립트의 전역 객체입니다. `screen` , `location` , `history` , `navigator` , `document` 는 `window` 의 속성으로 포함되며, 전역 객체 특성상 `window.` 는 보통 생략해서 사용합니다.

BOM 객체 실습에서는 다음 항목들이 다뤄졌습니다.

객체	핵심 속성/메소드	예시 동작
<code>screen</code>	<code>width</code> , <code>height</code> 등	화면 너비/높이 정보를 확인합니다.
<code>navigator</code>	<code>userAgent</code> 등	브라우저/환경 정보를 확인합니다.
<code>location</code>	<code>href</code> , <code>reload()</code>	URL 확인, 변경 및 새로고침을 수행합니다.
<code>history</code>	<code>back()</code> , <code>forward()</code> , <code>go(n)</code>	뒤로/앞으로 이동 및 스택 기반 이동을 수행합니다.
<code>window</code>	<code>open()</code> , <code>close()</code>	새 창 열기 및 창 닫기를 수행합니다.

타이머 함수는 `setTimeout` (지연 후 1회 실행)과 `setInterval` (주기적 반복 실행), `clearInterval` (반복 중지)가 소개되었습니다. 이 과정에서 함수가 “데이터처럼” 변수에 저장되거나, 함수에 인자로 전달될 수 있다는 자바스크립트의 함수 1급 시민 특성이 다시 연결되었습니다.

브라우저 API 문서 탐색 방법으로 W3Schools의 JS Reference 및 Mozilla MDN(Web API 문서)을 활용해 속성과 메소드를 확인하는 방법이 안내되었습니다.

핵심 키워드

#BOM

#DOM

#window

#location

#setTimeout

8. 문자열 백틱과 배열, JSON 기초

문자열은 홑따옴표, 쌍따옴표, 백틱을 사용할 수 있으며, 백틱은 변수 삽입과 멀티라인에 유리하고, 배열은 JS에서 핵심 컬렉션이며 JSON(객체 리터럴)은 키-값 구조로 다양한 타입을 담습니다.

문자열 표현은 홑따옴표, 쌍따옴표, 백틱(템플릿 리터럴) 3가지를 사용할 수 있습니다. 백틱은 `${변수}` 형태로 문자열 내 변수 치환이 가능하며, 여러 줄 문자열을 자연스럽게 작성할 수 있어 가독성과 템플릿 생성에 유리합니다.

백틱을 쓰지 않으면 문자열 결합을 `+` 로 수행해야 하며, 템플릿을 만들수록 가독성이 떨어지는 문제가 생깁니다. 백틱은 테이블 같은 HTML 템플릿 문자열 생성에서도 들여쓰기와 줄바꿈을 유지할 수 있어 프레임워크에서도 활용도가 높다고 정리되었습니다.

`String` 과 `Number` 는 내장 객체이며, MDN에서 생성자/스태틱 메소드/인스턴스 메소드를 구분해서 문서를 읽는 방법이 안내되었습니다. `Number` 에서는 `toString(16)` 같은 진법 변환, `toFixed` (소수점 자릿수 지정), `MAX_VALUE/MIN_VALUE` 같은 범위, `isNaN/isInteger` , `parseInt` (문자열 숫자 변환) 등이 예시로 언급되었습니다.

`Date` 는 일반적으로 `new Date()` 로 생성하며, 년/월/일/시/분/초 같은 값을 얻거나 설정하는 메소드가 제공된다는 흐름으로 정리되었습니다.

배열은 자바보다 간단히 `[]` 로 생성하며, 빈 배열도 `[]` 로 생성 가능합니다. 배열 접근은 인덱스로 수행합니다. 반복은 일반 `for` 외에도 `for...of` 와 `for...in` 이 소개되었고, `for...of` 는 값(value)을, `for...in` 은 인덱스(index)를 반환한다는 점이 핵심입니다.

배열 메소드로는 다음 항목들이 중점적으로 다뤄졌습니다.

분류	메소드	핵심 역할
끝 요소 조작	<code>push</code> , <code>pop</code>	마지막에 추가/마지막 요소 제거를 수행합니다.
중간 조작	<code>splice(start, deleteCount, ...items)</code>	삭제, 삽입을 한 메소드로 처리하며 옵션 인자가 존재합니다.
검색/부분	<code>indexOf</code> , <code>slice</code>	값 위치 검색 및 부분 배열 추출을 수행합니다.
변형	<code>reverse</code> , <code>fill</code>	역순 변환 및 특정 값으로 채우기를 수행합니다.
변환	<code>join</code>	배열 값을 구분자로 연결하여 문자열을 생성합니다.

자바와 달리 자바스크립트에서는 컬렉션보다 배열 사용 비중이 높다는 점이 강조되었으며, ES6의 Set/Map 이 존재하더라도 실무, 기존 코드 흐름에서 배열 중심으로 많이 사용된다는 맥락이 추가되었습니다.

객체 리터럴(중괄호 키-값)은 자바스크립트에서 "객체"로 자주 사용되며, 다른 이름으로 JSON 표현식 (JavaScript Object Notation)이라고 부릅니다. JSON의 값에는 기본형/참조형(배열, 또 다른 객체, 함수, 클래스 등)까지 폭넓게 담을 수 있으며, 중첩 구조도 가능하다는 점이 소개되었습니다.

핵심 키워드

#템플릿 리터럴

#백틱

#for...of

#splice

#JSON